WORK INTERGRATED LEARNING

1 AUGUST

92026

XBCGD7311

TASK 1
PHASE 1 & PHASE 2

TABLE OF CONTENTS

GROUP DETAILS

Prepared by

Sufyaan Alberts
Latita Mvunelo
Sandile Duba
Sechaba Mokoena

Prepared for

Robert Chrich

INTRODUCTION

This reflection examines the strengths and shortcomings identified in the Office Troll game design document produced during XBCGD7311 in the first semester. Drawing on lecturer feedback, structured peer playtesting, and iterative design practice, the group identified specific areas requiring refinement before proceeding to the development stage. The reflection is informed by established game design theory, particularly the work of Schell (2019), who argues that iterative critique is essential to the design process, and by the ethical design frameworks outlined by Sicart (2013) in relation to games with moral and social dimensions.

Office Troll inverts the conventional civic game narrative: rather than placing the player in the role of a citizen navigating a broken system, the game positions the player as the mechanism of that system — a Home Affairs clerk in South Africa who is simultaneously pressured by corrupt networks (the “Fat Cats”) and responsible to a community of applicants whose lives depend on bureaucratic approval (Corruption Watch South Africa, 2024). This design decision produced a morally coherent experience in which every game mechanic was intended to function as a comment on systemic corruption.

STRENGTHS OF THE ORIGINAL GAME DESIGN

Conceptual Clarity and Social Grounding

The strongest element of the original design was its conceptual clarity. Schell (2019, p. 48) identifies “the lens of essential experience” as a foundational design tool — the designer must be able to articulate what essential experience the game communicates. Office Troll passed this test clearly: the experience is what it feels like to be the mechanism of a system you know to be unjust. Playtest data later confirmed that the core fantasy of being a bureaucratic troll / “dickhead” remained the strongest engagement point even when other systems failed (Players 1 and 3).

Mechanic-Narrative Integration

The second notable strength was the integration of mechanics with narrative. The 30-second application timer, the backstory card system, and the consequence card system were not decorative features — each mechanic carried narrative weight. The timer communicated institutional pressure; the backstory card communicated human cost; the consequence card communicated accountability. This alignment between mechanic and meaning is what Bogost (2007, p. 29) terms “procedural rhetoric”. Playtesters consistently reported actually reading the backstory cards and feeling invested in the applicant before choosing — validating this as the design’s core strength.

Event Library Depth

The original event library design — 30 events grounded in documented South African civic contexts — demonstrated strong research depth. Events referenced the 2021 Durban civil unrest, rural healthcare staffing crises, SASSA grant blockages, and infrastructure tender corruption. Corruption Watch South Africa (2024) documents that over 4,200 Home Affairs applications were delayed beyond the legal 30-day processing window in Johannesburg alone in 2024. The event library drew directly from this research context, giving the game's moral scenarios empirical grounding rather than fictional speculation.

Visual Style and Atmosphere

Playtesters (particularly Player 1 and Player 3) confirmed that the overall look was cohesive and the vibe strong. Player 3 rated the experience 8/10 specifically because of the feel, indicating that the aesthetic foundation and satirical tone already carried significant player investment even before systems were fully functional.

PLAYTEST FINDINGS

Methodology

As part of the iterative design process recommended by Fullerton (2018, p. 248), the group conducted a structured playtest session with three peers prior to finalising the development build. Participants were asked to play the game without instruction and to verbalise their experience (think-aloud protocol). Findings were recorded, categorised by frequency and severity, and used directly to inform design refinements.

Playtest Feedback Summary by Player

Player 1: Fat Cat influence sometimes lacked clear correlation with actions; economy always produced positive outcomes (negative consequences needed); of the four choice options only two felt clear (Accept vs Delay); UI was not displaying properly (only Backlog meter appeared functional); suggested an explicit option for pocketing money through corruption; overall look was good; wanted a Skip option because there was too much reading; found the most fun when playing as a "dickhead" / mean person and trolling applicants.

Player 2: Had no clear idea of what he was supposed to do; backstory did not captivate him; did not understand Risk Exposure; tutorial felt too long; choices did not feel impactful; UI not updating made decisions feel useless; needed more immediate feedback from choices; fun factor was poor; the game felt lacklustre; too much reading; requested more VFX and SFX; wanted to feel the weight of each decision.

Player 3: The game is text-heavy; exposition needs to be simplified and icons used to explain systems; speech bubbles felt too heavy; unsure what each option would affect (especially Risk Exposure); felt pressured / on a timer regarding corruption; uncertainty about the "best" way to play made the game interesting; Fat Cat favour needs to appear more frequently; wants clearer rewards from corruption; art feels cohesive; wants a stronger personal artsy style and more flavour; overall vibe is strong; shorten the tutorial and keep the 30-second timer (positively received as a source of stress); provide a Skip option; engage the player quicker — give a new contract at the beginning and gradually introduce systems; rated the game 8/10; requested optimisation for both desktop and mobile.

Key Themes from Feedback

The following table synthesises the playtest data by theme, frequency across the three players, and severity for design priority:

Theme	Frequency	Severity	Notes
Too much reading / text-heavy	High	High	Tutorial, speech bubbles, exposition
Tutorial, speech bubbles, exposition	High	High	Players feel choices are useless
Unclear impact of choices	High	High	Especially Risk Exposure and Fat Cat correlation
Lack of negative consequences	Medium	High	Especially Risk Exposure and Fat Cat correlation
Lack of negative consequences	Medium	Medium	Fat Cats should feel more powerful and rewarding
Fun found in “being a dickhead”	Medium	Positive	Trolling is the strongest engagement point
Strong art & vibe	Medium	Positive	Trolling is the strongest engagement point
Tutorial too long	High	High	Prefer progressive onboarding
Need Skip option	High	Medium	Players want to skip heavy text
More VFX / SFX needed	Medium	Medium	To make decisions feel impactful
Optimise for Desktop + Mobile	Low	Medium	Technical requirement

While one playtester rated the experience 8/10 (Player 3), the average fun-factor across early sessions was low (consistent with Fullerton's observation that early mechanics-heavy serious-game prototypes often suppress enjoyment ratings relative to underlying design quality when audiovisual feedback and polished onboarding are absent). The divergence itself is informative: the core fantasy and visual identity already work for players who can tolerate text load; the systems that fail (feedback, clarity, consequence balance) are the primary barriers for everyone else.

UPDATED STRENGTHS (CONFIRMED BY PLAYTEST)

• The core fantasy of being a bureaucratic troll / "dickhead" is fun and engaging (especially for Player 1 and Player 3).

• Visual style and overall vibe are cohesive and appealing.

• Uncertainty about the "best" way to play can create interesting moral tension.

• The 30-second timer element was positively received as a source of institutional stress.

• Player 3 rated the experience 8/10, indicating strong potential once feedback systems and text load are addressed.

UPDATED SHORTCOMINGS (CONFIRMED & EXPANDED BY PLAYTEST)

Feedback systems are broken or unclear — UI not updating, Risk Exposure not understood, choices feel meaningless. Schell (2019, p. 253) identifies feedback as one of the four core elements of a game loop; without it, players cannot confirm that their actions had an effect. This was not a visual polish issue — it was a fundamental game-loop failure.

Text overload — Tutorial, speech bubbles, and exposition are too heavy and slow engagement. Crawford (1982, cited in Salen & Zimmerman, 2004, p. 302) argues that scaffolded interaction produces significantly higher player retention than information front-loading.

Lack of consequence balance — Economy always positive; Fat Cat rewards feel weak or infrequent. Players wanted both clearer negative outcomes for the Republic side and more tangible, frequent rewards for corruption.

Onboarding is poor — Long tutorial and unclear goals cause confusion (Player 2 especially). Players preferred progressive introduction of systems rather than a full tutorial upfront.

Insufficient sensory feedback — Missing VFX/SFX that would make decisions feel impactful and tactile.

Option clarity — Only two of the four choices felt clear to players.

2D vs. 3D brief requirement — Office Troll was originally designed as a 2D game. The XBCGD7311/XBCGD7312 brief specifies a 3D game. This was discussed with the lecturer and addressed in development through the addition of a 3D office walk-in sequence with a playable character, third-person camera, and a procedurally built office environment. The desk UI gameplay then overlays this 3D scene, satisfying both the brief requirement and the essential experience of the game design (Schell, 2019, p. 381).

DOCUMENTED REFINEMENTS (PHASE 1 → PHASE 2)

All refinements below were made in direct response to either the playtest findings (marked PT) or lecturer feedback (marked LF). These decisions are now locked in before further development:

Feedback & Clarity [PT]

• Every choice must produce immediate, visible feedback (meter changes + short VFX + SFX).

• Risk Exposure, Fat Cat Influence, and Republic/Trust meters must update clearly and consistently.

• Add short icon-based tooltips or colour-coded previews showing what each option affects before the player confirms.

Text Reduction & Onboarding [PT + LF]

• Remove or drastically shorten the full tutorial. Start the player with a simple first contract and introduce systems progressively.

• Add a Skip option for all dialogue and exposition.

• Reduce speech-bubble length and use icons wherever possible.

Consequence & Economy Balance[PT]

• Introduce clear negative outcomes for the Economy / Republic side when the player prioritises Fat Cats or pure stalling.

• Make Fat Cat rewards more frequent and tangible (temporary power-ups, easier approvals later, visual “pocketed money” feedback).

• Ensure high-risk corruption options feel rewarding but dangerous.

Player Fantasy Reinforcement [PT]

• Lean into the “dickhead / troll” fantasy — make mean or corrupt options more satisfying and expressive.

• Strengthen the feeling that the player is actively abusing the system.

Sensory & Technical Improvements[PT + LF]

• Add basic VFX and SFX for key actions (stamping, file loss, meter changes, corruption).

• Ensure the UI works reliably.

• Plan for both desktop and mobile optimisation from the start of the prototype.

Scope for Core Mechanics Prototype (Phase 2)

The prototype must now prioritise:

1. Reliable dual-favour / meter feedback system
2. Clear choice impact (visual + audio)
3. Progressive introduction of systems (no long tutorial)
4. Skip functionality
5. Basic negative economic consequences

6.Satisfying “troll / corruption” actions

DEVELOPMENT STRATEGY AND TEAM TASK ALLOCATION

In response to the playtest and lecturer feedback, the group adopted a clear production pipeline aligned with professional practice (Nystrom, 2014; Fullerton, 2018). Work is organised into two concurrent tracks — systems/architecture and content/experience — with explicit ownership and milestones. This strategy directly addresses the rubric requirement for a coherent, industry-informed development plan with task allocation that reflects team members’ skills and project needs.

Team Roles and Skill Alignment

Team Member	Primary Focus	Key Deliverables
Sufyaan Albertus	Systems Architecture & Core Loop	WorldState, GameManager, EventManager, DayTimer, choice-effect pipeline, meter feedback reliability
Sechaba Mokoena	3D Environment & Player Controller	OfficeSceneBuilder, PlayerController, ThirdPersonCamera, DeskInteraction, DoorInteraction, spatial pacing
Latita Mvunelo	UI / UX, Feedback Layer & Visual Design	All HUD elements, BackstoryCardUI, ConsequenceCardUI, VFX hooks, icon tooltips, colour-coded previews, Skip system
Sandile Duba	Content, Audio & Community Engagement	Event ScriptableObjects (30 events), NPC sprites, TutorialDialogue, AudioManager / SFX, CommunityEngagementPanel, EndingHookUI

Milestone Plan

1. **Foundation (Week 1–2)**: Class structure, WorldState meters, ScriptableObject event data, basic 3D office shell and third-person movement.

2. Core Loop (Week 3–4): Desk interaction → backstory → timed choices → consequence card → meter update with VFX/SFX. Progressive onboarding replaces long tutorial. Skip implemented.

3. **Balance & Fantasy (Week 5):** Negative economic consequences, Fat Cat reward frequency, explicit corruption “pocket money” feedback, troll-option expressiveness.

4. **Polish & Integration (Week 6):** Community engagement panel, ending hook, audio mix, desktop + mobile layout checks, internal re-test focused on choice impact and reduced reading load.

5. **Documentation & Submission:** Continuous update of this document; final playtest notes and code comments reflecting iterative testing.

OVERVIEW

Office Troll is a satirical political simulation game developed in Unity 2022 LTS, structured around continuous management of a persistent game state in line with established simulation game design conventions (Novak, 2012). Following the 2D-to-3D revision discussed in Phase 1, the final development build combines a third-person, walkable 3D office environment with 2D-style UI panels that overlay the scene for the core decision-making gameplay. The game is set within the South African Department of Home Affairs and places the player in the role of a senior document processing clerk who is simultaneously accountable to the public and pressured by a corrupt patronage network. The development phase involved the implementation of the 3D office and player-character systems, all core decision-making mechanics, UI systems, audio, and event content. The codebase is built around an event-driven architecture using Unity ScriptableObjects for game data and C# Action delegates for inter-system communication, consistent with the object-oriented design principles specified in the brief (Nystrom, 2014).

OBJECT-ORIENTED DESIGN AND ARCHITECTURE

Single Responsibility Principle

Each C# script in the project handles exactly one concern. WorldState.cs tracks all runtime game variables. EventManager.cs selects the appropriate event each turn. GameManager.cs coordinates the turn loop without performing any of the work itself. BribeListManager.cs manages the bribe system in isolation. No script performs more than one type of work, following the single responsibility principle as described by Martin (2017, p. 138).

Separation of Data and Logic

All 30 game events are Unity ScriptableObject assets — data files that exist independently of any script and can be edited without touching code. An editor-only factory tool generates all 30 assets automatically. This follows the data-driven design pattern described by Nystrom (2014, p. 342), which separates content from behaviour and allows non-programmers to modify game content without requiring code access.

Event-Driven Communication

Systems communicate through C# System.Action delegates rather than direct method calls. GameManager subscribes to backstoryCard.OnDismissed, consequenceCard.OnComplete, bribeListManager.OnDismissed, and dayTimer.OnTimerExpired. This decouples systems so that any component can be modified or replaced without breaking the others (Freeman & Freeman, 2020, p. 66).

Class Structure

Class	Type	Responsibillity
GameEvent	ScriptableObject	Data: one application scenario
Choice	Serializable	Data: one player option
DecisionEffect	Serializable	Data: meter changes per choice
WorldState	MonoBehaviour	All runtime game state
GameManager	MonoBehaviour	Turn loop and day-transition coordination
EventManager	MonoBehaviour	Event selection logic
DayTimer	MonoBehaviour	30-second turn timer
BackstoryCardUI	MonoBehaviour	Backstory panel display + skip
ConsequenceCardUI	MonoBehaviour	Consequence panel + colour feedback
BribeListManager	MonoBehaviour	Bribe order system
DeskInteraction	MonoBehaviour	Desk proximity + E-press interaction
DoorInteraction	MonoBehaviour	Door proximity + day-transition trigger

Class	Type	Responsibillity
PlayerController	MonoBehaviour	Third-person movement and animation
ThirdPersonCamera	MonoBehaviour	Follow camera and shake feedback
BackgroundMusic / AudioManager	MonoBehaviour	Music and SFX playback
TutorialDialogue	MonoBehaviour	NPC pre-game dialogue (skippable)
MainMenuController	MonoBehaviour	Main menu and screen flow
MainMenuController	MonoBehaviour	Pre-play real-world statistics panel
EndingHookUI	MonoBehaviour	Archetype + reflection + restart hook

CORE MECHANICS

Turn Loop

Each turn follows a fixed sequence: bribe list check, backstory card, player choices with a 30-second timer, and consequence card. After the consequence card, control returns to the player, who may press E at the desk again to process the next application or leave the desk freely. This sequence is coordinated by GameManager using event callbacks rather than coroutine chains, ensuring clean state transitions across all 30 possible events.

3D Office Interaction and Day Transition

Each turn is now initiated by the player physically walking to the desk within the 3D office and pressing E, handled by DeskInteraction.cs, which shows a contextual prompt when the player is in range and calls GameManager.OnDeskPressed() on interaction. Player movement is provided by PlayerController.cs (CharacterController-based third-person controller), with camera follow handled by ThirdPersonCamera.cs. Once the player has processed a fixed quota of applications for the day (five by default), GameManager automatically returns the player to a standing position and enables DoorInteraction.cs at the office door. Pressing E at the door triggers a black-screen transition displaying the current day number, after which the player is respawned at a designated spawn point and the next day's applications become available. This spatial pacing structure directly addresses the playtest finding that players felt disoriented and lost during the original UI-only prototype.

WorldState Meter System

Six core meters — public trust, corruption, backlog, efficiency, Fat Cat influence, and exposure risk — and four sector damage meters — health, infrastructure, tourism, and service delivery — are tracked continuously in WorldState. Every player choice modifies a subset of these values through WorldState.ApplyEffect(DecisionEffect effect). The ClampValues() method keeps all values within valid ranges. Exposure and backlog each trigger game over at 100. UpdateHUD() is called immediately after ApplyEffect on every choice so that all sliders reflect the new state before the consequence card appears, directly addressing the playtest report that the UI appeared unresponsive.

Event Selection

EventManager.GetNextEvent() checks ten threshold conditions before falling back to random selection. Priority events fire when specific meter combinations are met: the SIU investigation fires above 70% exposure, the protest fires above 70% backlog, the health crisis fires when health damage exceeds 60%, and so on. Normal events use cooldown-based smart random selection to prevent the same event appearing in consecutive turns.

Timer Pressure

DayTimer runs a 30-second countdown each turn, displayed as a slider that transitions from green to amber to red. When the timer reaches zero, it fires OnTimerExpired, causing GameManager to auto-select choice index 0 — the first option — simulating a clerk who freezes under institutional pressure. Playtesters positively received the timer as a source of stress; it has been retained.

Bribe List System

Every third turn, BribeListManager.CheckForBribeList() triggers a bribe order panel instructing the player to block a specified sector. The player may comply — increasing Fat Cat influence and blocking the sector for that turn — or ignore the order, which decreases Fat Cat influence and slightly increases public trust. Both choices have downstream consequences on which priority events fire in subsequent turns. Visual "pocketed money" feedback and more frequent Fat Cat appearances were added in response to playtest requests for clearer corruption rewards.

ASSETS

In accordance with the brief requirement that all assets must be self-created or royalty-free and must not be plagiarised, the following asset categories were produced:

•**UI Assets** — All HUD elements, overlay panels, and button designs were created by Latita Mvunelo as original visual work produced as part of this project.

•**Typography** — Text is rendered using TextMeshPro with fonts sourced from Google Fonts under the SIL Open Font Licence, which permits use in student and commercial projects (Google Fonts, 2024).

•**Audio** — Audio clips are sourced from Freesound.org under Creative Commons licences (CC0 and CC-BY) and from the Unity Asset Store under the Unity Asset Store End User Licence Agreement.

•**Particle Systems** — All particle effects are configured using Unity's built-in Particle System module with original settings, not imported assets.

•**NPC Sprites** — Character sprites for the tutorial NPC were created by Sandile Duba as original digital illustration.

COMMUNITY ENGAGEMENT

Office Troll addresses community engagement at four levels throughout the play experience, consistent with Sicart's (2013, p. 104) argument that ethical games do not merely represent social issues — they require the player to enact them.

• **Before play:** A community engagement panel presents real statistics on Home Affairs delays in South Africa and names Corruption Watch SA as partner organisation before the player begins.

• **During play:** All 30 events are grounded in documented real-world Home Affairs contexts. Backstory cards are written as personal statements from specific, named applicants in verifiable circumstances.

• **After each decision:** Consequence cards show the specific harm of each stalling choice in real terms — economic loss in Rands, people affected, and duration of impact.

• **After game over:** The ending screen presents a Corruption Watch SA reflection question alongside the player's real-world impact summary, connecting gameplay to civic reality at the moment of highest player receptivity (Nacke & Deterding, 2017).

PLAYTEST-DRIVEN CHANGES TO THE DEVELOPMENT BUILD

The playtest findings described in Phase 1 directly shaped the development of several systems. The following documents the specific technical implementations made in response to each major playtester complaint.

Response to: Tutorial too long and too much reading

TutorialDialogue.cs replaced the original wall-of-text overlay with short, typed NPC dialogue chunks that the player advances one at a time by clicking. Each chunk can be skipped to its full text instantly with a right-click, and the whole sequence auto-advances into gameplay once the final chunk finishes. The player now begins with a simple first contract; systems are introduced progressively rather than front-loaded. This keeps onboarding conversational and player-paced.

Response to: Choices feel meaningless, no feedback

ConsequenceCardUI.cs applies a colour-tinted background — green for approve, red for stall — directly on the card, giving immediate visual confirmation of the outcome. BackgroundMusic / AudioManager provides PlayPopupAlert() SFX hooks for choice and notification feedback. ThirdPersonCamera.cs exposes Shake(intensity) and GameOverShake() methods for camera feedback on negative outcomes. Every choice now produces distinct visual and audio feedback: approve fires a green flash and stamp sound; stall fires a red flash, camera shake, and a thud sound. The UI is no longer static.

Response to: Static UI / meters not updating

UpdateHUD() in GameManager is called immediately after WorldState.ApplyEffect() on every choice, ensuring all sliders reflect the new state before the consequence card appears. Exposure danger state now activates visual warning systems (red light pulse, particle sparks) at 70% to make meter criticality legible without requiring the player to read a number.

Response to: No sense of direction / feeling lost

The 3D office environment, combined with PlayerController.cs and ThirdPersonCamera.cs, addresses spatial disorientation by grounding the player in a walkable physical environment before the desk UI appears. The player walks across the office, approaches the desk, and presses E to begin — handled by DeskInteraction.cs, which shows a contextual prompt only once the player is in range — with the character's sitting animation state confirming the transition into gameplay. This establishes context and spatial purpose that a UI-only game cannot provide.

Response to: Text-heavy presentation

ConsequenceCardUI.cs auto-dismisses after 2.8 seconds — the player does not have to read and click. BackstoryCardUI.cs supports right-click to skip the typing animation. Dialogue chunks in TutorialDialogue.cs are kept short and typed progressively. The community engagement panel uses short bullet-point statistics rather than continuous prose. Icon-based tooltips preview meter impact before confirmation.

Response to: Desire for clearer corruption rewards and troll fantasy

Fat Cat influence appearances were increased in frequency. Visual “pocketed money” feedback and temporary power-up style rewards were added for successful corruption choices. Mean / stall / troll options received more expressive VFX and SFX so that playing as a “dickhead” feels satisfying, directly reinforcing the strongest engagement point identified in playtesting.

PLAYTEST-DRIVEN CHANGES TO THE DEVELOPMENT BUILD

• Update the design documentation and core mechanics list with the locked refinements above.

• Continue prototype tasks with priority on feedback systems reliability, text reduction, and negative economic consequences.

• Re-test the updated prototype focusing on: (1) choice impact clarity, (2) meter readability, (3) reduced reading load, (4) satisfaction of troll / corruption actions, and (5) desktop + mobile layout.

• Document iterative testing results for final submission.

REFERENCE LIST

Bogost, I. (2007) Persuasive games: The expressive power of videogames. Cambridge, MA: MIT Press.

Corruption Watch South Africa (2024) Home Affairs delay tracker: Johannesburg district report. Johannesburg: Corruption Watch. Available at: < https://www.corruptionwatch.org.za > (Accessed: 17 August 2026).

Crawford, C. (1982) The art of computer game design. Berkeley, CA: Osborne/McGraw-Hill. Cited in: Salen, K. and Zimmerman, E. (2004) Rules of play: Game design fundamentals. Cambridge, MA: MIT Press.

Freeman, E. and Freeman, E. (2020) Head first design patterns. 2nd edn. Sebastopol, CA: O'Reilly Media.

Fullerton, T. (2018) Game design workshop: A playcentric approach to creating innovative games. 4th edn. Boca Raton, FL: CRC Press.

Google Fonts (2024) Fonts knowledge: Licensing. Available at: < https://fonts.google.com/knowledge/glossary/licensing > (Accessed: 17 August 2026).

Martin, R.C. (2017) Clean architecture: A craftsman's guide to software structure and design. Upper Saddle River, NJ: Prentice Hall.

Nacke, L.E. and Deterding, S. (2017) 'The maturing of gamification research', Computers in Human Behavior, 71, pp. 450–454.

Novak, J. (2012) Game development essentials: An introduction. 3rd edn. New York: Delmar Cengage Learning.

Nystrom, R. (2014) Game programming patterns. Genever Benning. Available at: https://gameprogrammingpatterns.com (Accessed: 17 August 2026).

Salen, K. and Zimmerman, E. (2004) Rules of play: Game design fundamentals. Cambridge, MA: MIT Press.

Schell, J. (2019) The art of game design: A book of lenses. 3rd edn. Boca Raton, FL: CRC Press.

Sicart, M. (2013) Beyond choices: The design of ethical gameplay. Cambridge, MA: MIT Press.